```
import asyncio
import logging
from aiogram import Bot, Dispatcher, types, F
from aiogram.filters import Command
from aiogram.types import InlineKeyboardMarkup, InlineKeyboardButton
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
import aiohttp
import os

# ==================== НАСТРОЙКИ ====================
BOT_TOKEN = os.getenv("BOT_TOKEN", "ВАШ_ТОКЕН_ОТ_BOTFATHER")
GROQ_API_KEY = os.getenv("GROQ_API_KEY", "ВАШ_КЛЮЧ_GROQ")
ADMIN_ID = int(os.getenv("ADMIN_ID", "ВАШ_TELEGRAM_ID"))  # Твой ID, чтобы тебе приходили вопросы

# ==================== ПРОМПТ ====================
SYSTEM_PROMPT = """Ты — Блог-Наставник, персональный ИИ-ассистент эксперта Нади.

Твоя задача — помогать клиенту развивать блог, создавать контент (особенно Reels), выстраивать систему и двигаться к результату.

Правила общения:
- Отвечай коротко, по делу и структурированно.
- Всегда учитывай профиль клиента (если он есть в контексте).
- Давай конкретные действия, хуки, структуры и примеры.
- Тон: поддерживающий, прямой, без воды и лести.
- Если вопрос сложный или требует личного мнения Нади — предложи написать ей через кнопку.
- В конце ответа при необходимости предлагай следующий шаг.
"""

# ==================== ИНИЦИАЛИЗАЦИЯ ====================
logging.basicConfig(level=logging.INFO)
bot = Bot(token=BOT_TOKEN)
dp = Dispatcher()

class AskNadia(StatesGroup):
    waiting_for_question = State()

# ==================== КЛАВИАТУРЫ ====================
def main_keyboard():
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="Спросить у Нади", callback_data="ask_nadia")],
        [InlineKeyboardButton(text="Идея для Reels", callback_data="reels_idea")],
        [InlineKeyboardButton(text="Контент-план на неделю", callback_data="content_plan")]
```

```
    ])

# ==================== AI ФУНКЦИЯ ====================
async def ask_groq(user_message: str, user_context: str = "") -> str:
    url = "https://api.groq.com/openai/v1/chat/completions"
    headers = {
        "Authorization": f"Bearer {GROQ_API_KEY}",
        "Content-Type": "application/json"
    }

    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": f"{user_context}\n\nВопрос клиента: {user_message}"}
    ]

    data = {
        "model": "llama-3.1-8b-instant",  # быстрая и бесплатная модель
        "messages": messages,
        "temperature": 0.7,
        "max_tokens": 1000
    }

    async with aiohttp.ClientSession() as session:
        async with session.post(url, headers=headers, json=data) as resp:
            if resp.status == 200:
                result = await resp.json()
                return result["choices"][0]["message"]["content"]
            else:
                return "Извини, сейчас не могу ответить. Попробуй чуть позже или спроси
Надю."

# ==================== ХЕНДЛЕРЫ ====================
@dp.message(Command("start"))
async def cmd_start(message: types.Message):
    await message.answer(
        "Привет! Я — твой Блог-Наставник.\n\n"
        "Могу помочь с идеями для Reels, контент-планом, чек-листами и ответить на
вопросы по блогу.\n\n"
        "Просто напиши вопрос или выбери действие ниже:",
        reply_markup=main_keyboard()
    )

@dp.callback_query(F.data == "ask_nadia")
async def ask_nadia_start(callback: types.CallbackQuery, state: FSMContext):
    await callback.message.answer("Напиши свой вопрос. Я передам его Наде.")
    await state.set_state(AskNadia.waiting_for_question)
    await callback.answer()
```

```
@dp.message(AskNadia.waiting_for_question)
async def process_nadia_question(message: types.Message, state: FSMContext):
    question = message.text
    user = message.from_user

    # Отправляем вопрос тебе
    await bot.send_message(
        ADMIN_ID,
        f"Новый вопрос от клиента:\n\n"
        f"Имя: {user.full_name}\n"
        f"Username: @{user.username}\n"
        f"ID: {user.id}\n\n"
        f"Вопрос:\n{question}"
    )

    await message.answer(
        "Вопрос отправлен Наде. Она ответит, как только сможет.\n\n"
        "А пока можешь задать мне другой вопрос или попросить идею для Reels.",
        reply_markup=main_keyboard()
    )
    await state.clear()

@dp.callback_query(F.data == "reels_idea")
async def reels_idea(callback: types.CallbackQuery):
    prompt = "Дай одну сильную идею для Reels. Укажи тему, хук (первые 3 секунды),
краткую структуру и рекомендацию по визуалу. Ниша клиента пока не указана —
сделай универсальную, но сильную."
    answer = await ask_groq(prompt)
    await callback.message.answer(answer, reply_markup=main_keyboard())
    await callback.answer()

@dp.callback_query(F.data == "content_plan")
async def content_plan(callback: types.CallbackQuery):
    prompt = "Составь контент-план на 7 дней в виде таблицы. Колонки: День,
Платформа, Тема, Формат, Хук. Сделай акцент на Reels."
    answer = await ask_groq(prompt)
    await callback.message.answer(answer, reply_markup=main_keyboard())
    await callback.answer()

@dp.message(F.text)
async def handle_message(message: types.Message):
    # Все обычные сообщения идут в ИИ
    thinking = await message.answer("Думаю...")
    answer = await ask_groq(message.text)
    await thinking.delete()
    await message.answer(answer, reply_markup=main_keyboard())

# ==================== ЗАПУСК ====================
```

```
async def main():
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())
```